PEI 1: dinámica de actitud y teorema del eje intermedio

La carpeta `PEI_1` explicada paso a paso: física, métodos numéricos y código

AM1 – Grupo 1 (MUSE, UPM)

25 de octubre de 2026

Resumen

Este documento recorre la carpeta `PEI_1` poco a poco. Primero, **qué pide el enunciado**. Después, **la física** (ecuaciones de Euler, invariantes, geometría de Poinsot, separatriz y análisis espectral) y **los métodos numéricos** (EE, EI, CN, RK4, el nuevo Leapfrog, Newton, Richardson y la actitud). Luego, el **código**: la versión base, que está a medio hacer, y la carpeta `Solucion_PEI1`, la versión completa. Las GUI se explican por bloques, no línea a línea. Al final hay experimentos numéricos, la lista de **cambios hechos en el código** y una lista de **preguntas probables del tribunal** con su respuesta. El código ya está corregido: los listados de este documento son los de la versión corregida, y cada recuadro rojo explica qué fallaba y cómo se arregló. Todos los números y figuras salen de ejecutar el código de la carpeta. El teorema en sí está explicado con más calma en `Teorema_eje_intermedio_explicado.pdf`.

Índice

1. Qué hay en la carpeta y qué pide el enunciado **3**
1.1. El enunciado en pocas palabras . . . 3
1.2. Mapa de la carpeta . . . 3

2. La física, paso a paso **5**
2.1. Paso 1: las ecuaciones de Euler . . . 5
2.2. Paso 2: las dos cantidades que no cambian . . . 5
2.3. Paso 3: la geometría de Poinsot . . . 5
2.4. Paso 4: la cota $2I_1T \le L^2 \le 2I_3T$ . . . 5
2.5. Paso 5: la separatriz . . . 6
2.6. Paso 6: equilibrios, jacobiano y autovalores . . . 7
2.7. Paso 7: Explorer 1 y la regla del eje mayor . . . 7

3. Los métodos numéricos, paso a paso **9**
3.1. Los cuatro de siempre . . . 9
3.2. Leapfrog (LF): el método nuevo . . . 9
3.3. Qué significa $|R|$ en cada eje . . . 10
3.4. Newton con jacobiana exacta . . . 10
3.5. Richardson y el orden . . . 10
3.6. RK4 adaptativo (RK4A): el paso lo decide el error . . . 11
3.7. La actitud: hacia dónde apunta el satélite . . . 11

4. La versión base: `PEI_1/*.py` **13**
4.1. `space_physics.py` . . . 13
4.2. `numerical_engine.py` . . . 14
4.3. `gui_reactive.py` . . . 15

5. La solución: `PEI_1/Solucion_PEI1` **16**
5.1. `space_physics.py`: la receta 16
5.2. `numerical_engine.py`: el cocinero 18
5.3. `cuerpos.py`: el catálogo de cuerpos 20
5.4. `gui_reactive.py`: la GUI sencilla 21
5.5. `gui_actitud.py`: el cuadro de mando completo 23
5.6. `test_solution.py`: las pruebas automáticas 25

6. Experimentos con el caso por defecto **27**

7. Frente al enunciado y cambios hechos en el código **28**
7.1. Lo que pide el enunciado 28
7.2. Cambios hechos en el código 28

8. Preguntas probables del tribunal **29**

1. Qué hay en la carpeta y qué pide el enunciado

1.1. El enunciado en pocas palabras

`PEI1.pdf` pide tres cosas:

1. **Analizar** la rotación libre de un sólido rígido: ecuaciones de Euler, las dos cantidades que se conservan, la geometría de Poinsot y por qué el eje intermedio es inestable (linealización).

2. **Simularla** con varios esquemas numéricos (Euler, Crank–Nicolson, RK4), estimar el error con Richardson, estudiar las regiones de estabilidad $|R(z)| \leq 1$ y vigilar cuánto se desvían las cantidades que deberían conservarse.

3. **Programarlo en tres capas separadas**, con una GUI interactiva: deslizadores para I_1, I_2, I_3, Ω y Δt; cambio de integrador; espacio de fases 3D con las superficies de Poinsot y la separatriz; y animación 3D del satélite.

La nota se reparte a partes iguales (25 % cada uno) entre rigor físico, métodos numéricos, software y GUI, y defensa oral individual.

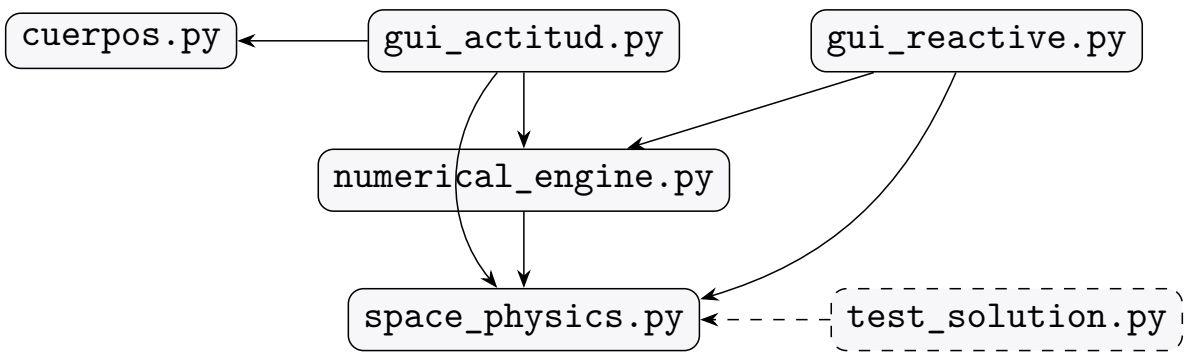
Analogía: las tres capas son un restaurante

- **Física** (`space_physics.py`) es *la receta*: dice qué ingredientes hay y cómo se combinan (las ecuaciones de Euler), pero no cocina.
- **Motor numérico** (`numerical_engine.py`) es *el cocinero*: sabe técnicas (EE, RK4...) que valen para cualquier receta.
- **GUI** (`gui_reactive.py`) es *la sala*: el cliente elige (deslizadores, botones) y ve el plato (gráficas), sin entrar en la cocina.

"Desacoplar" significa que puedes cambiar de cocinero sin cambiar la receta, y redecorar la sala sin tocar la cocina.

1.2. Mapa de la carpeta

Archivo	Capa	Estado
`PEI1.pdf`	–	enunciado
`space_physics.py`	física	versión base, corregida
`numerical_engine.py`	métodos	versión base, corregida
`gui_reactive.py`	GUI	versión base, apenas empezada (10 líneas)
`Solucion_PEI1/space_physics.py`	física	completa
`Solucion_PEI1/numerical_engine.py`	métodos	completa, con paso adaptativo
`Solucion_PEI1/cuerpos.py`	geometría	catálogo de 6 cuerpos con su inercia
`Solucion_PEI1/gui_reactive.py`	GUI	GUI sencilla
`Solucion_PEI1/gui_actitud.py`	GUI	cuadro de mando completo (unas 1150 líneas)
`Solucion_PEI1/test_solution.py`	pruebas	13 tests automáticos
`Solucion_PEI1/README.md`	–	cómo ejecutar



Las flechas significan “importa a”. La física no importa a nadie, los métodos solo importan la física, y las GUI importan las dos. Es justo la separación en capas que pide el enunciado.

Problema encontrado: una errata del enunciado

En la ecuación (2) de `PEI1.pdf`, la tercera línea de la izquierda pone $I_3\dot{\omega}_1$ y debería ser $I_3\dot{\omega}_3$. La forma despejada de la derecha sí es correcta.

2. La física, paso a paso

2.1. Paso 1: las ecuaciones de Euler

En ejes del cuerpo, alineados con los ejes principales ($I_1 < I_2 < I_3$), y sin pares externos:

$$\dot{\omega}_1 = \frac{I_2 - I_3}{I_1}\,\omega_2\omega_3, \qquad \dot{\omega}_2 = \frac{I_3 - I_1}{I_2}\,\omega_3\omega_1, \qquad \dot{\omega}_3 = \frac{I_1 - I_2}{I_3}\,\omega_1\omega_2. \tag{1}$$

Qué significan:

- $\boldsymbol{\omega} = (\omega_1, \omega_2, \omega_3)$ es la rapidez de giro alrededor de cada eje del cuerpo.
- El momento angular $\mathbf{L} = (I_1\omega_1, I_2\omega_2, I_3\omega_3)$ está quieto en el espacio. Pero los ejes del cuerpo giran, así que, visto desde el cuerpo, $\boldsymbol{\omega}$ cambia. Es como mirar una farola desde un tiovivo.
- Cada componente cambia según el producto de las otras dos. Por eso el sistema es **no lineal**.

Conexión con los Milestones

Es un problema de Cauchy $\dot{\mathbf{U}} = \mathbf{F}(\mathbf{U})$ con $\mathbf{U} = \boldsymbol{\omega}$. Todo lo de los Milestones (esquemas, Richardson, estabilidad) se aplica tal cual: solo cambia la función $\mathbf{F}$.

2.2. Paso 2: las dos cantidades que no cambian

$$2T = I_1\omega_1^2 + I_2\omega_2^2 + I_3\omega_3^2, \qquad L^2 = I_1^2\omega_1^2 + I_2^2\omega_2^2 + I_3^2\omega_3^2.$$

Demostración en una línea, derivando y sustituyendo (1):

$$\begin{aligned} \tfrac{d}{dt}(2T) &= 2\omega_1\omega_2\omega_3\big[(I_2 - I_3) + (I_3 - I_1) + (I_1 - I_2)\big] = 0, \\ \tfrac{d}{dt}L^2 &= 2\omega_1\omega_2\omega_3\big[I_1(I_2 - I_3) + I_2(I_3 - I_1) + I_3(I_1 - I_2)\big] = 0. \end{aligned}$$

Físicamente: no hay rozamiento, así que no se pierde energía; no hay pares externos, así que el tamaño del momento angular no cambia.

2.3. Paso 3: la geometría de Poinsot

En el espacio $(\omega_1, \omega_2, \omega_3)$, cada ley de conservación es un **elipsoide**:

- Energía: $I_1\omega_1^2 + I_2\omega_2^2 + I_3\omega_3^2 = 2T$, con semiejes $\sqrt{2T/I_k}$.
- Momento angular: $I_1^2\omega_1^2 + I_2^2\omega_2^2 + I_3^2\omega_3^2 = L^2$, con semiejes L/I_k.

El estado tiene que estar en los dos a la vez, así que se mueve por su **intersección**, una curva cerrada llamada **polhodia** (Fig. 1).

2.4. Paso 4: la cota $2I_1T \le L^2 \le 2I_3T$

El cociente $L^2/2T$ es una **media ponderada** de los momentos de inercia:

$$\frac{L^2}{2T} = \frac{I_1\,(I_1\omega_1^2) + I_2\,(I_2\omega_2^2) + I_3\,(I_3\omega_3^2)}{(I_1\omega_1^2) + (I_2\omega_2^2) + (I_3\omega_3^2)},$$

con pesos $I_k\omega_k^2 \ge 0$. Una media siempre está entre el valor más pequeño y el más grande, así que $I_1 \le L^2/2T \le I_3$.

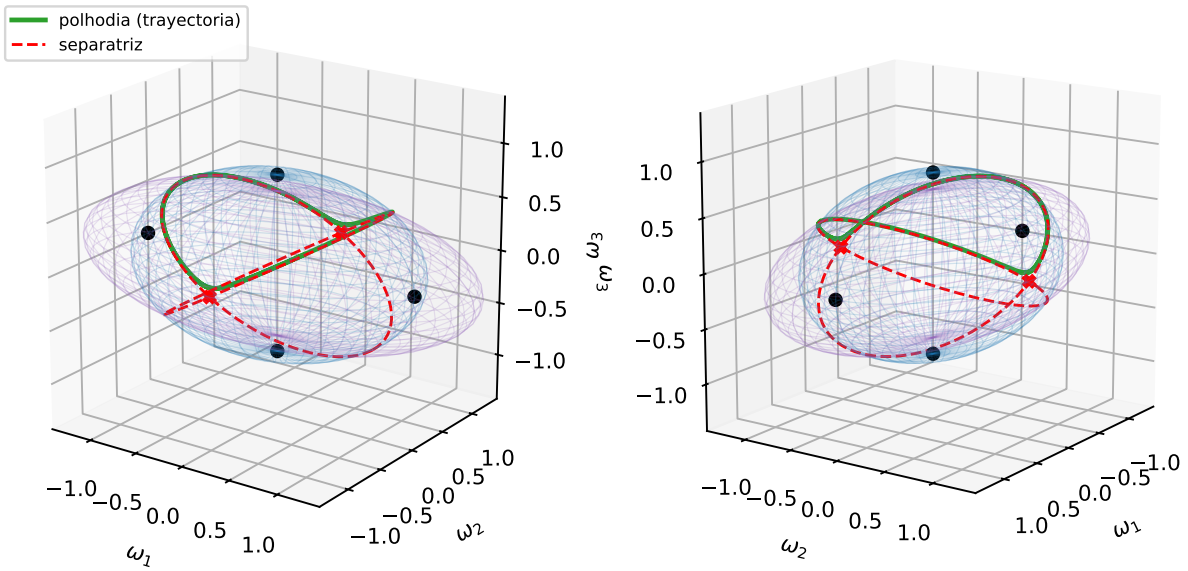


Figura 1: Los dos elipsoides, la trayectoria real (verde, calculada con `ecuaciones_rotacion` de la solución) y la separatriz (rojo, calculada con `separatriz` de la solución). Puntos negros: giros puros estables ($\pm e_1$, $\pm e_3$). X rojas: giros puros alrededor del eje intermedio ($\pm e_2$), donde se cruzan las dos ramas de la separatriz.

Analogía: la nota media

La nota media de un curso nunca puede ser más baja que tu peor nota ni más alta que la mejor. $L^2/2T$ es la "inercia media" con la que gira el cuerpo, y por eso está entre I_1 e I_3. Si un programa da unos invariantes fuera de esa cota, hay un error en los datos o en el código. Eso es lo que la GUI comprueba con el ✓.

Además, esa inercia media dice de qué familia es el movimiento:

- $L^2/2T < I_2$: el cuerpo da vueltas alrededor de e_1;
- $L^2/2T > I_2$: el cuerpo da vueltas alrededor de e_3;
- $L^2/2T = I_2$: está justo en la frontera, la **separatriz**.

2.5. Paso 5: la separatriz

Imponer $L^2 = 2I_2T$ y restar las dos leyes de conservación elimina ω_2:

$$L^2 - 2I_2T = I_1(I_1 - I_2)\,\omega_1^2 + I_3(I_3 - I_2)\,\omega_3^2 = 0 \quad \Longrightarrow \quad \omega_3 = \pm\, p\,\omega_1, \qquad p = \sqrt{\frac{I_1(I_2 - I_1)}{I_3(I_3 - I_2)}}.$$

Son **dos planos** que contienen al eje ω_2. Al cortarlos con el elipsoide de energía salen dos elipses que se cruzan en $\pm e_2$ (rojo en la Fig. 1).

Analogía: la divisoria de aguas

En una cordillera, la línea de cumbres separa el agua que va a un mar de la que va al otro. Una gota que cae justo encima se queda en equilibrio, pero el mínimo soplo la manda a un lado. La separatriz separa los movimientos "alrededor de e_1" de los "alrededor de e_3". Girar alrededor de e_2 es estar justo en la cima de esa divisoria.

2.6. Paso 6: equilibrios, jacobiano y autovalores

Los giros puros $\boldsymbol{\omega}^* = (\Omega, 0, 0)$, $(0, \Omega, 0)$ y $(0, 0, \Omega)$ son equilibrios: con dos componentes nulas, todos los productos de (1) se anulan. Para saber si son estables, se mira qué hace una perturbación pequeña con la **jacobiana** $J = \partial \mathbf{F}/\partial \boldsymbol{\omega}$:

$$J(\boldsymbol{\omega}) = \begin{pmatrix} 0 & \frac{I_2 - I_3}{I_1}\omega_3 & \frac{I_2 - I_3}{I_1}\omega_2 \\ \frac{I_3 - I_1}{I_2}\omega_3 & 0 & \frac{I_3 - I_1}{I_2}\omega_1 \\ \frac{I_1 - I_2}{I_3}\omega_2 & \frac{I_1 - I_2}{I_3}\omega_1 & 0 \end{pmatrix}.$$

Sus autovalores en cada equilibrio:

Giro en	Autovalores	Tipo	Comportamiento
e_1	$0,\ \pm i\,\Omega\sqrt{\frac{(I_3-I_1)(I_2-I_1)}{I_2 I_3}}$	centro	la perturbación oscila
e_2	$0,\ \pm\,\Omega\sqrt{\frac{(I_3-I_2)(I_2-I_1)}{I_1 I_3}}$	**silla**	**la perturbación crece**
e_3	$0,\ \pm i\,\Omega\sqrt{\frac{(I_3-I_2)(I_3-I_1)}{I_1 I_2}}$	centro	la perturbación oscila

- **Imaginarios puros** ($\pm i\omega$): es el oscilador armónico de los Milestones; la perturbación da vueltas sin crecer. Se llama **centro**.
- **Reales, uno positivo** ($\pm\sigma$): la perturbación crece como $e^{\sigma t}$. Se llama **punto de silla**, igual que un puerto de montaña.
- **El autovalor** 0: el enunciado no lo menciona, pero existe. Corresponde a cambiar un poco la velocidad Ω *a lo largo del mismo eje*. Eso lleva a otro giro puro, que también es equilibrio, así que la perturbación ni crece ni decrece. Importa para la GUI (sección 5.5).

Lo que dice la simulación: la raqueta de la GUI

La raqueta de `cuerpos.py` tiene $I = (0.211,\ 1.793,\ 2.000)$ y la GUI empieza con $\Omega = 2.5$. Con `estabilidad_ejes`: e_1: $\pm 2.221\,i$; e_2: ± 2.203; e_3: $\pm 2.474\,i$. En el eje intermedio la perturbación crece como $e^{2.2t}$, multiplicándose por 10 cada 1.05 s. Por eso la raqueta voltea muy deprisa: ω_2 cambia de signo en $t = 3.70,\ 9.78,\ 15.86,\ 21.94,\ 28.01$ s.

Un matiz para la defensa. Para la silla (e_2), la linealización basta: el teorema de Hartman–Grobman dice que, si ningún autovalor tiene parte real nula, el sistema no lineal se comporta cerca del equilibrio como el lineal. Para los centros (e_1, e_3) *no* basta, porque sus autovalores tienen parte real nula y el teorema no se aplica. Lo que garantiza que e_1 y e_3 sean estables en el sistema no lineal son **las dos leyes de conservación**: cerca de e_1 y e_3 las polhodias son curvas cerradas pequeñas, así que el estado no puede alejarse. Es un detalle que un tribunal puede preguntar.

2.7. Paso 7: Explorer 1 y la regla del eje mayor

El enunciado cita el satélite Explorer 1 (1958). Se diseñó para girar alrededor de su eje de *menor* inercia, y acabó dando volteretas alrededor del de *mayor* inercia. Fijado L, la energía es $2T = L_1^2/I_1 + L_2^2/I_2 + L_3^2/I_3$. Esa energía es **máxima** girando sobre e_1 (I menor) y **mínima** girando sobre e_3 (I mayor). Las antenas flexibles de Explorer 1 vibraban y disipaban energía, pero sin cambiar L, porque no hay par externo. Así que el satélite fue bajando de energía hasta el mínimo: el giro sobre e_3.

Analogía

Una pelota en una montaña con rozamiento acaba en el fondo del valle. Con disipación, la única rotación estable a largo plazo es la del eje de **mayor** inercia: es el "valle" de energía. Sin disipación (el modelo del PEI), los ejes 1 y 3 son estables y solo el 2 es inestable.

3. Los métodos numéricos, paso a paso

3.1. Los cuatro de siempre

Esquema	Fórmula de un paso	Orden	$R(z)$ (factor de amplificación)
EE	$\mathbf{U}^{n+1} = \mathbf{U}^n + \Delta t\,\mathbf{F}(\mathbf{U}^n)$	1	$1+z$
EI	$\mathbf{U}^{n+1} = \mathbf{U}^n + \Delta t\,\mathbf{F}(\mathbf{U}^{n+1})$	1	$1/(1-z)$
CN	$\mathbf{U}^{n+1} = \mathbf{U}^n + \frac{\Delta t}{2}(\mathbf{F}(\mathbf{U}^n) + \mathbf{F}(\mathbf{U}^{n+1}))$	2	$(1+z/2)/(1-z/2)$
RK4	cuatro "miradas" $k_1..k_4$ con pesos 1-2-2-1	4	$1+z+\frac{z^2}{2}+\frac{z^3}{6}+\frac{z^4}{24}$

Son los de los Milestones. El factor de amplificación $R(z)$, con $z = \Delta t\,\lambda$, dice por cuánto se multiplica una perturbación en un paso. El código lo implementa en `estabilidad`.

3.2. Leapfrog (LF): el método nuevo

$$\boxed{\mathbf{U}^{n+1} = \mathbf{U}^{n-1} + 2\Delta t\,\mathbf{F}(\mathbf{U}^n)}$$

Salta de t_{n-1} a t_{n+1} usando la pendiente del punto del medio, t_n. Es la **diferencia centrada**.

Analogía: el salto de rana

Una rana salta por encima de la piedra central y usa la inclinación de esa piedra para decidir el salto. Euler explícito usaba la pendiente del punto de salida, que está en un extremo. Usar la del centro es mucho más equilibrado, y por eso LF es de **orden 2** siendo tan barato como Euler explícito: una sola evaluación de $\mathbf{F}$ por paso.

- **Es de dos pasos**: necesita $\mathbf{U}^{n-1}$ y $\mathbf{U}^n$. El primer paso hay que darlo con otro método. La versión base usa EE y la solución usa RK4, que es más preciso.
- **Sus factores de amplificación**: al aplicarlo a $\dot{y} = \lambda y$ sale $r^2 - 2zr - 1 = 0$, así que $r = z \pm \sqrt{1+z^2}$. Hay **dos** raíces: una física ($\approx e^z$) y otra *parásita* ($\approx -e^{-z}$) que la solución exacta no tiene.
- **Su región de estabilidad es solo el segmento** $[-i, i]$ del eje imaginario. Ahí las dos raíces tienen módulo exactamente 1.
  - Para *centros* (oscilaciones sin rozamiento) con $\Delta t|\lambda| \le 1$, LF conserva la amplitud como CN, pero es explícito y barato.
  - Para problemas con rozamiento ($\mathrm{Re}\, z < 0$), la raíz parásita crece y LF explota.
- Es **simétrico en el tiempo**: si se cambia Δt por $-\Delta t$ la fórmula es la misma. Los métodos simétricos no tienen deriva sistemática de la energía (se ve en la sección 6).

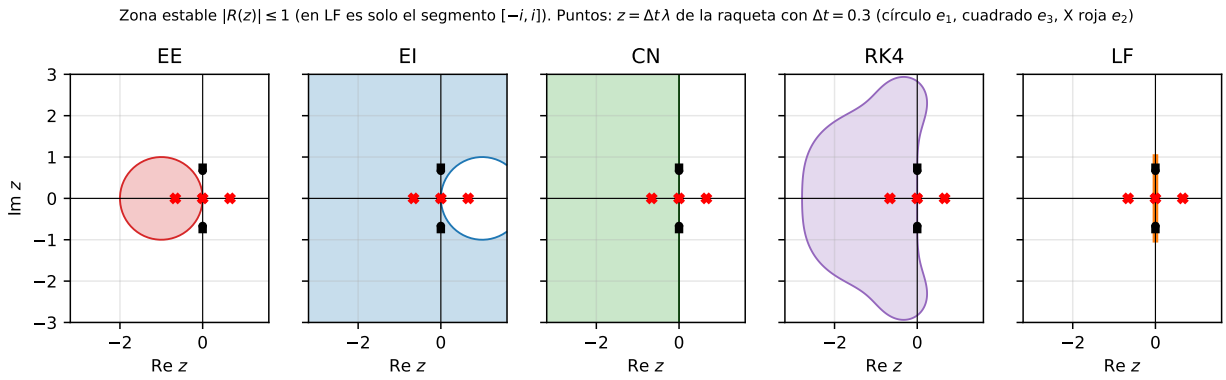


Figura 2: Regiones de estabilidad de los cinco esquemas de paso fijo (RK4A comparte la de RK4). Los puntos son $z = \Delta t\lambda$ para la raqueta de la GUI, con $\Delta t = 0.3$ (exagerado para que se vean). Los centros e_1 y e_3 están sobre el eje imaginario y la silla e_2 (X roja) sobre el eje real; la X de la derecha está fuera de *todas* las regiones. Eso no es un fallo de los métodos: la solución física crece, y un método que la reprodujera bien *debe* crecer.

3.3. Qué significa $|R|$ en cada eje

Lo que dice la simulación: `stability_report` con la raqueta, $\Omega = 2.5$, $\Delta t = 0.01$

Esquema	e_1 (centro)	e_2 (silla)	e_3 (centro)
EE	1.000247 (amplifica)	1.022028	1.000306 (amplifica)
EI	disipa: 0.999753 (*)	1.022525	disipa: 0.999694 (*)
CN	1 (conserva)	1.022274	1 (conserva)
RK4	1 (conserva a 10^{-13})	1.022273	1 (conserva)
LF	1 (conserva)	1.022271	1 (conserva)
exacto	1	$e^{\sigma\Delta t} = 1.022273$	1

- En los centros lo correcto es $|R| = 1$. EE se pasa (crea energía), EI se queda corto (la disipa), CN y LF clavan el 1, y RK4 está a 10^{-13} de él.
- En la silla lo correcto es crecer exactamente $e^{\sigma\Delta t}$. Todos crecen, y RK4 coincide en 7 cifras.

(*) Valores calculados quitando el autovalor 0, como hace ahora la GUI. Antes de la corrección la GUI daba 1.0 en esas casillas: ver sección 5.5.

3.4. Newton con jacobiana exacta

EI y CN obligan a resolver una ecuación en cada paso. Se usa Newton, como en los Milestones, con una mejora: la solución usa la **jacobiana analítica** (`jacobiano`) en lugar de aproximarla con diferencias finitas. Es más precisa (no hay error de h) y más barata (no evalúa $\mathbf{F}$ tres veces más).

Lo que dice la simulación

Media de iteraciones de Newton por paso con la raqueta: **1.47** en EI y **1.29** en CN. El problema no es lineal, así que no siempre basta un salto como en el oscilador, pero la propuesta inicial (Euler explícito) ya cae muy cerca y Newton converge cuadráticamente.

3.5. Richardson y el orden

Es lo de `Cauchy_Problem.py` del Milestone 3: resolver con N, $2N$ y $4N$ pasos, medir el orden con $q = \log_2(d_1/d_2)$ y estimar el error del fino como (grueso − fino)/$(2^q - 1)$.

Analogía

Medir la mesa con tres reglas cada vez más finas. Lo que cambia la medida al afinar la regla dice cuánto te equivocabas y a qué ritmo mejoras, sin conocer la medida verdadera.

3.6. RK4 adaptativo (RK4A): el paso lo decide el error

El enunciado pide esquemas "de paso simple y adaptativo". Con paso fijo hay que escoger Δt pensando en el peor momento, que aquí es el volteo, cuando $\boldsymbol{\omega}$ cambia muy deprisa. Ese Δt pequeño se desperdicia en las mesetas, donde casi nada cambia. Un método adaptativo da pasos largos donde puede y cortos donde debe.

Analogía: conducir por una carretera de montaña

En las rectas vas deprisa y en las curvas frenas. No conduces toda la carretera a la velocidad que exige la curva más cerrada.

`Solucion_PEI1/numerical_engine.py` implementa `RK4A` con **duplicación del paso**, que es la misma idea de Richardson aplicada a un solo paso:

1. Se da un subpaso h con RK4 ("grueso") y el mismo tramo en dos mitades $h/2$ ("fino").
2. Como RK4 es de orden 4, (fino − grueso)/15 estima el error local, porque $2^4 - 1 = 15$.
3. Si ese error es menor que la tolerancia (10^{-9} relativa), se acepta el subpaso y se suma la corrección (extrapolación de Richardson). Si no, se repite con un h menor.
4. El siguiente h se escala con $(\text{tolerancia}/\text{error})^{1/5}$, porque el error local de RK4 va como h^5. Se multiplica además por 0.9 para dejar margen, y nunca se cambia en más de un factor 5 hacia abajo o 2 hacia arriba.

`RK4A` recorre así el intervalo $[t, t + \Delta t]$ con los subpasos que haga falta y devuelve el estado final. Por eso se usa igual que los demás esquemas: el Δt de la GUI pasa a ser solo el intervalo entre fotogramas.

Lo que dice la simulación

Raqueta, $\Omega = 2.5$, hasta $t = 30$, comparado con la referencia:

	error final de $\boldsymbol{\omega}$	máxima deriva de T	tiempo
RK4, $\Delta t = 0.01$	1.1×10^{-8}	2.1×10^{-9}	0.03 s
RK4, $\Delta t = 0.1$	1.6×10^{-4}	9.6×10^{-5}	0.004 s
RK4A, $\Delta t = 0.1$	3.7×10^{-7}	1.6×10^{-8}	0.04 s
RK4A, $\Delta t = 1$	3.8×10^{-7}	1.4×10^{-8}	0.02 s

Con RK4A el error ya no depende de Δt, sino de la tolerancia: con $\Delta t = 0.1$ o con $\Delta t = 1$ sale lo mismo. Por eso el botón de Richardson dice "no aplica a RK4A": al dividir Δt entre 2 el error no cambia, así que no hay un orden que medir.

3.7. La actitud: hacia dónde apunta el satélite

Las ecuaciones de Euler dan $\boldsymbol{\omega}$, pero no la orientación. La solución la guarda como una **matriz de rotación** $R(t)$, cuyas columnas son los ejes del cuerpo vistos desde el espacio, y la avanza así (`paso_actitud`):

1. toma la velocidad media del paso, $\bar{\boldsymbol{\omega}} = (\boldsymbol{\omega}^n + \boldsymbol{\omega}^{n+1})/2$;
2. gira el cuerpo *exactamente* un ángulo $\theta = |\bar{\boldsymbol{\omega}}|\Delta t$ alrededor del eje $\bar{\boldsymbol{\omega}}$, con la fórmula de Rodrigues $Q = I + \sin\theta\, S + (1 - \cos\theta) S^2$;
3. compone: $R^{n+1} = R^n\, Q$.

Analogía: girar con un transportador

La versión con cuaterniones y Euler explícito de la carpeta de la raqueta "empuja" la orientación en línea recta y luego tiene que normalizar (devolverla a la esfera). Rodrigues, en cambio, *gira* de verdad en cada paso, como quien usa un transportador. El producto de rotaciones es siempre una rotación, así que la matriz nunca se deforma.

Lo que dice la simulación

Con la raqueta y RK4 hasta $t = 30$: $R^T R$ se separa de la identidad en 7×10^{-15} como mucho y $\det R = 1$ a 2×10^{-15}, o sea, una rotación perfecta. Frente a una referencia muy precisa, la orientación se desvía $0.018°$, $0.032°$ y $0.053°$ en $t = 10$, 20 y 30.

4. La versión base: `PEI_1/*.py`

Son los tres archivos de la raíz de `PEI_1`: un primer intento con la estructura de tres capas. Tal como estaban, **no funcionaban** (errores comprobados ejecutándolos). Ya están corregidos; los listados son los de la versión corregida.

4.1. `space_physics.py`

```
1  import numpy as np
2  from numpy import array
3
4  def sort_I (I_1:float, I_2:float, I_3:float):
5      I1=min(I_1, I_2, I_3)
6      I3=max(I_1, I_2, I_3)
7      I2=I_1 + I_2 + I_3 - I1 - I3
8      return I1, I2, I3
9
10 def ecuaciones_rotacion(W:array, I1:float, I2:float, I3:float):
11     # W = [w1, w2, w3]: devuelve [dw1/dt, dw2/dt, dw3/dt] (ecuaciones de Euler).
12     dW1=(I2-I3)/I1*W[1]*W[2]
13     dW2=(I3-I1)/I2*W[2]*W[0]
14     dW3=(I1-I2)/I3*W[0]*W[1]
15     return array([dW1, dW2, dW3])
16
17 def invariantes(W:array, I1:float, I2:float, I3:float):
18     # energia = 2T y momento = L^2. Deben cumplir 2*I1*T <= L^2 <= 2*I3*T.
19     energia=(I1*W[0]**2 + I2*W[1]**2 + I3*W[2]**2)
20     momento=(I1*W[0])**2 + (I2*W[1])**2 + (I3*W[2])**2
21     tolerancia=1e-12*momento
22     if momento < I1*energia - tolerancia or momento > I3*energia + tolerancia:
23         print("Los invariantes no son consistentes, revise los valores de entrada.")
24     return energia, momento
```

- **Líneas 4–8, `sort_I`**: ordena las inercias para que $I_1 \leq I_2 \leq I_3$. El del medio se calcula como suma total menos mínimo menos máximo. Estaba bien.
- **Líneas 10–15, `ecuaciones_rotacion`**: las ecuaciones de Euler (1). Recibe $(\omega_1, \omega_2, \omega_3)$ y devuelve un vector *nuevo* con las derivadas.
- **Líneas 17–24, `invariantes`**: calcula $2T$ y L^2, comprueba la cota $2I_1T \leq L^2 \leq 2I_3T$ y los devuelve.

Problema encontrado: lo que estaba mal (ya corregido)

1. **Índices desplazados**. En Python, $\omega_1, \omega_2, \omega_3$ son `W[0], W[1], W[2]`, pero el código usaba `W[1], W[2], W[3]` y escribía en `W[3..5]`: `IndexError: index 3 is out of bounds`.
2. **Productos cambiados**: dos de las tres ecuaciones multiplicaban las componentes equivocadas.
3. **Sobrescribía el vector de entrada** en vez de devolver uno nuevo. Un esquema que llama varias veces a **F** (RK4) habría usado un estado estropeado.
4. `invariantes` comparaba $2I_1T$ con $\sqrt{L^2} = L$ en lugar de con L^2: para un estado válido como $\boldsymbol{\omega} = (0.1, 1, 0.1)$ e $I = (1, 2, 3)$ decía "no son consistentes". Además no devolvía nada. Ahora compara con L^2, con una pequeña tolerancia para el redondeo.
5. Las dos primeras líneas añadían la carpeta del repositorio al `sys.path` solo para la importación rota de `numerical_engine.py`. Se han quitado junto con ella.

4.2. numerical_engine.py

```python
import numpy as np
from numpy import array
from numpy.linalg import norm
from typing import Callable

def partition(t0: float, tf: float, N: float):
    # Crea una partición del intervalo [t0, tf] en N subintervalos equiespaciados.
    return np.array([t0 + i * (tf - t0) / N for i in range(N + 1)])

def sol_error_cauchy(
    esquema: str,
    U0: np.ndarray,
    N_t: int,
    t0: float,
    tf: float,
    F: Callable[[np.ndarray, float], np.ndarray],
):
    coarse = integrar_Cauchy(esquema, U0, N_t, t0, tf, F)
    fine = integrar_Cauchy(esquema, U0, 2 * N_t, t0, tf, F)
    finer = integrar_Cauchy(esquema, U0, 4 * N_t, t0, tf, F)

    difference_coarse_fine = np.linalg.norm(coarse - fine[::2])
    difference_fine_finer = np.linalg.norm(fine[::2] - finer[::4])
    orden = np.log(difference_coarse_fine / difference_fine_finer) / np.log(2)
    error = error_richardson(coarse, fine[::2], orden)
    return coarse, error, orden


def integrar_Cauchy(
    esquema: str,
    U0: np.ndarray,
    N_t: int,
    t0: float,
    tf: float,
    F: Callable[[np.ndarray, float], np.ndarray],
):
    metodos = {"EE": EE, "EI": EI, "CN": CN, "RK4": RK4, "LF": LF}
    U = np.zeros((N_t + 1, len(U0)))
    U[0, :] = U0
    t = partition(t0, tf, N_t)
    metodo = metodos[esquema]
    if esquema == "LF":
        # Para el esquema Leapfrog, necesitamos un paso inicial con otro método.
        # Con RK4 (y no EE) el primer paso no excita el modo parásito de LF.
        U[1, :] = RK4(U[0, :], t[1] - t[0], t[0], F)
        for n in range(1, N_t):
            U[n + 1, :] = metodo(U[n, :], t[n + 1] - t[n], t[n], F, U[n - 1, :])
    else:
        for n in range(N_t):
            U[n + 1, :] = metodo(U[n, :], t[n + 1] - t[n], t[n], F)
    return U
```

- Es el motor del Milestone 3 copiado en un solo archivo (EE, EI, CN, RK4, Newton, jacobiana numérica, Richardson), más el esquema nuevo **LF**.
- **Líneas 42–47**: para LF, el primer paso se da con RK4 y luego se pasa también `U[n-1]` al método.
- La función LF (líneas 97–99) es la fórmula del salto de rana tal cual:

```python
def LF(U:array, At:float, t:float, F:Callable[[array,float], array], U_prev:array):
    # Esquema Leapfrog (salto de rana) de segundo orden.
    return U_prev + 2 * At * F(U, t)
```

Problema encontrado: lo que estaba mal (ya corregido)

1. **El archivo no se podía importar.** La línea `from Milestone_3 import Integradores` cargaba `Milestone_3/Integradores.py`, que a su vez hace `from Esquemas_temporales import ...`. Esa importación solo funciona si la carpeta `Milestone_3` está en el `sys.path`, y aquí no lo estaba: `ModuleNotFoundError`. Además `Integradores` no se usaba, porque el archivo define su propio `integrar_Cauchy`. Se ha quitado la línea.
2. **LF arrancaba con un paso de Euler explícito**, y el orden medido con Richardson salía 2.43 en vez de 2. Ese primer paso, poco preciso, excita la raíz parásita de LF, cuyo error cambia de signo en cada paso y confunde la medida del orden. Arrancando con RK4, como en la solución, sale 2.00.

4.3. `gui_reactive.py`

```python
import space_physics,numerical_engine
#inputs:
# Hacen falta tres inercias distintas para que exista eje intermedio
# (con I_1 = I_2 el cuerpo es simetrico, como una moneda, y no voltea).
# Ademas, en un solido real I_3 <= I_1 + I_2.
I_1=0.010
I_2=0.014
I_3=0.020

I1, I2, I3 = space_physics.sort_I(I_1, I_2, I_3)
```

Solo ordena tres inercias; la GUI de verdad es la de la solución. Los valores originales eran curiosos: $I = (0.014, 0.014, 0.028)$.

Idea clave: esos valores eran una moneda

Dos inercias iguales ($I_1 = I_2$) significan que el cuerpo es **simétrico** alrededor de e_3, y $I_3 = I_1 + I_2$ significa que es **plano**. Un cuerpo plano y simétrico es un disco: una moneda. Una moneda **no tiene eje intermedio**, así que no hay efecto Dzhanibekov que ver. Se han cambiado por $(0.010, 0.014, 0.020)$: tres inercias distintas y con $I_3 \leq I_1 + I_2$, como debe cumplir un sólido real.

5. La solución: `PEI_1/Solucion_PEI1`

5.1. `space_physics.py`: la receta

```python
def ecuaciones_rotacion(W, I1, I2, I3):
    return np.array(
        [
            (I2 - I3) / I1 * W[1] * W[2],
            (I3 - I1) / I2 * W[2] * W[0],
            (I1 - I2) / I3 * W[0] * W[1],
        ]
    )


def jacobiano(W, I1, I2, I3):
    w1, w2, w3 = W
    return np.array(
        [
            [0, (I2 - I3) / I1 * w3, (I2 - I3) / I1 * w2],
            [(I3 - I1) / I2 * w3, 0, (I3 - I1) / I2 * w1],
            [(I1 - I2) / I3 * w2, (I1 - I2) / I3 * w1, 0],
        ]
    )


def invariantes(W, I1, I2, I3):
    energia = 0.5 * (I1 * W[0] ** 2 + I2 * W[1] ** 2 + I3 * W[2] ** 2)
    momento = (I1 * W[0]) ** 2 + (I2 * W[1]) ** 2 + (I3 * W[2]) ** 2
    return energia, momento


def historial_invariantes(U, I1, I2, I3):
    energia = 0.5 * (I1 * U[:, 0] ** 2 + I2 * U[:, 1] ** 2 + I3 * U[:, 2] ** 2)
    momento = (I1 * U[:, 0]) ** 2 + (I2 * U[:, 1]) ** 2 + (I3 * U[:, 2]) ** 2
    return energia, momento
```

- **`ecuaciones_rotacion`** (6–13): $\mathbf{F}$ de (1), con los índices bien y devolviendo un vector nuevo.
- **`jacobiano`** (16–24): la matriz J del paso 6, escrita a mano. La usan Newton y el análisis espectral.
- **`invariantes`** (27–30): devuelve T (¡no $2T$: lleva el 0.5) y L^2.
- **`historial_invariantes`** (33–36): lo mismo, pero para toda la tabla de estados de golpe. Sirve para dibujar la deriva.

```python
def estabilidad_ejes(I1, I2, I3, omega=1):
    equilibrios = {
        "I1": np.array([omega, 0, 0]),
        "I2": np.array([0, omega, 0]),
        "I3": np.array([0, 0, omega]),
    }
    jacobianos = {
        nombre: jacobiano(W, I1, I2, I3)
        for nombre, W in equilibrios.items()
    }
    autovalores = {
        nombre: np.linalg.eigvals(J)
        for nombre, J in jacobianos.items()
    }
    return equilibrios, jacobianos, autovalores


```

```
def separatriz(I1, I2, I3, energia, puntos=160):
    # Elipse completa en el plano w3 = pendiente * w1 (L^2 = 2 I2 T); la otra
    # rama es su simetrica respecto a w3 = 0.
    pendiente = np.sqrt(I1 * (I2 - I1) / (I3 * (I3 - I2)))
    w1_max = np.sqrt(2 * energia / (I1 + I3 * pendiente**2))
    w2_max = np.sqrt(2 * energia / I2)
    phi = np.linspace(0, 2 * np.pi, puntos)
    w1 = w1_max * np.cos(phi)
    return np.column_stack((w1, w2_max * np.sin(phi), pendiente * w1))
```

- **`estabilidad_ejes`** (39–53): para cada giro puro calcula la jacobiana y sus autovalores (`np.linalg.eigvals`). Es el paso 6 hecho por el ordenador.
- **`separatriz`** (56–64): la elipse del plano $\omega_3 = p\,\omega_1$ del paso 5. Recorre un ángulo ϕ con $\omega_1 = a\cos\phi$ y $\omega_2 = b\sin\phi$, y escoge a y b para que la curva esté sobre el elipsoide de energía. La otra rama ($\omega_3 = -p\,\omega_1$) se obtiene cambiando el signo de la tercera columna.

```
def paso_actitud(R, W, W_nuevo, dt):
    w = (W + W_nuevo) / 2
    modulo = np.linalg.norm(w)
    if modulo == 0:
        return R
    theta = modulo * dt
    wx, wy, wz = w / modulo
    S = np.array([[0, -wz, wy], [wz, 0, -wx], [-wy, wx, 0]])
    Q = np.eye(3) + np.sin(theta) * S + (1 - np.cos(theta)) * S @ S
    return R @ Q


def actitud(U, t):
    R = np.zeros((len(t), 3, 3))
    R[0] = np.eye(3)
    for n in range(len(t) - 1):
        R[n + 1] = paso_actitud(R[n], U[n], U[n + 1], t[n + 1] - t[n])
    return R


def estabilidad(esquema, z):
    z = np.asarray(z)
    if esquema == "EE":
        return 1 + z
    if esquema == "EI":
        return 1 / (1 - z)
    if esquema == "CN":
        return (1 + z / 2) / (1 - z / 2)
    if esquema in ("RK4", "RK4A"):
        # RK4A da subpasos de RK4 de tamano <= At; aqui se evalua con el maximo At.
        return 1 + z + z**2 / 2 + z**3 / 6 + z**4 / 24
    if esquema == "LF":
        return z + np.sqrt(1 + z**2)
```

- **`paso_actitud`** (67–76): la rotación de Rodrigues de la sección 3.7. `S` es la matriz que hace "producto vectorial por el eje".
- **`actitud`** (79–84): aplica `paso_actitud` a toda una simulación.
- **`estabilidad`** (87–99): el factor de amplificación $R(z)$ de cada esquema. Para LF devuelve solo la raíz física. Para RK4A usa el de RK4, porque cada subpaso es un paso de RK4.

5.2. `numerical_engine.py`: el cocinero

```python
def EE(U, At, t, F):
    return U + At * F(U, t)


def EI(U, At, t, F, J):
    return My_Newton(U, At, t, F, J)


def CN(U, At, t, F, J):
    def G(X, time):
        return (F(U, t) + F(X, time)) / 2

    def JG(X, time):
        return J(X, time) / 2

    return My_Newton(U, At, t, G, JG)


def RK4(U, At, t, F):
    k1 = F(U, t)
    k2 = F(U + At * k1 / 2, t + At / 2)
    k3 = F(U + At * k2 / 2, t + At / 2)
    k4 = F(U + At * k3, t + At)
    return U + At * (k1 + 2 * k2 + 2 * k3 + k4) / 6
```

- **EE, RK4**: idénticos a los de los Milestones.
- **EI, CN** reciben además `J`, la jacobiana exacta. CN vuelve a usar el truco de los Milestones: se disfraza de EI con la regla auxiliar `G`, cuya jacobiana es `J/2`.

```python
def RK4A(U, At, t, F, tol=1e-9, h_min=1e-10):
    """RK4 con paso adaptativo dentro de [t, t + At] (duplicacion del paso).

    Cada subpaso h se da entero y en dos mitades. Por Richardson (orden 4),
    (fino - grueso) / 15 estima el error local: si es menor que la tolerancia
    se acepta el subpaso (con la extrapolacion) y si no se repite con h menor.
    """
    t_final = t + At
    h = At
    while t_final - t > 1e-12 * max(1.0, abs(t_final)):
        h = min(h, t_final - t)
        if h < h_min * At:
            raise RuntimeError("RK4A: el paso adaptativo se ha hecho demasiado pequeno.")
        grueso = RK4(U, h, t, F)
        fino = RK4(RK4(U, h / 2, t, F), h / 2, t + h / 2, F)
        error = norm(fino - grueso) / 15
        limite = tol * (1 + norm(U))
        if error <= limite:
            U = fino + (fino - grueso) / 15
            t += h
        # Error local ~ h^5: nuevo paso con margen de seguridad 0.9.
        h *= min(2.0, max(0.2, 0.9 * (limite / max(error, 1e-300)) ** 0.2))
    return U


def LF(U, At, t, F, U_prev):
    return U_prev + 2 * At * F(U, t)


def My_Newton(U, At, t, F, J, tol=1e-10, max_iter=50):
```

```
    Y = U + At * F(U, t)
    R = Y - U - At * F(Y, t + At)
    iteraciones = 0
    while norm(R) > tol:
        iteraciones += 1
        if iteraciones > max_iter:
            raise RuntimeError(f"Newton no convergio en {max_iter} iteraciones.")
        matrix = np.eye(len(U)) - At * J(Y, t + At)
        Y = Y - np.linalg.solve(matrix, R)
        R = Y - U - At * F(Y, t + At)
    return Y
```

- **RK4A** (38–60): el paso adaptativo de la sección 3.6. `grueso` es un subpaso entero y `fino` dos mitades; su diferencia entre 15 es el error. Si el paso se hace absurdamente pequeño (el esquema diverge), lanza un error que la GUI recoge como "EL ESQUEMA DIVERGE".
- **LF** (63–64): el salto de rana, igual que en la versión base.
- **My_Newton** (67–78): propuesta con Euler; mientras el fallo $\|\mathbf{R}\|$ sea mayor que 10^{-10}, salto de Newton con la jacobiana exacta; aviso si no converge en 50 intentos.

```
def paso_Cauchy(esquema, U, U_prev, At, t, F, J):
    metodos = {"EE": EE, "EI": EI, "CN": CN, "RK4": RK4, "RK4A": RK4A, "LF": LF}
    if esquema == "LF":
        return LF(U, At, t, F, U_prev)
    if esquema in ("EI", "CN"):
        return metodos[esquema](U, At, t, F, J)
    return metodos[esquema](U, At, t, F)


def integrar_Cauchy(esquema, U0, N_t, t0, tf, F, J):
    U = np.zeros((N_t + 1, len(U0)))
    t = np.linspace(t0, tf, N_t + 1)
    U[0] = U0
    At = t[1] - t[0]

    if esquema == "LF":
        U[1] = RK4(U[0], At, t[0], F)
        for n in range(1, N_t):
            U[n + 1] = paso_Cauchy(
                esquema, U[n], U[n - 1], At, t[n], F, J
            )
    else:
        for n in range(N_t):
            U[n + 1] = paso_Cauchy(esquema, U[n], None, At, t[n], F, J)
    return t, U


def richardson_error(esquema, U0, N_t, t0, tf, F, J):
    t, coarse = integrar_Cauchy(esquema, U0, N_t, t0, tf, F, J)
    _, fine = integrar_Cauchy(esquema, U0, 2 * N_t, t0, tf, F, J)
    _, finer = integrar_Cauchy(esquema, U0, 4 * N_t, t0, tf, F, J)

    difference_1 = norm(coarse - fine[::2])
    difference_2 = norm(fine[::2] - finer[::4])
    order = np.log(difference_1 / difference_2) / np.log(2)
    error = (coarse - fine[::2]) / (2**order - 1)
    return t, coarse, error, order


def stability_report(esquema, eigenvalues, At):
    z = At * np.asarray(eigenvalues, dtype=complex)
```

```
    if z.size == 0:
        return True, 1.0
    if esquema == "LF":
        roots = np.stack((z + np.sqrt(1 + z**2), z - np.sqrt(1 + z**2)))
        maximum = np.max(np.abs(roots))
    else:
        maximum = np.max(np.abs(estabilidad(esquema, z)))
    # Tolerancia: |R| = 1 exacto (CN, LF en el eje imaginario) puede salir
    # 1 + 2e-16 por redondeo.
    return maximum <= 1 + 1e-12, maximum
```

- **`paso_Cauchy`** (81–87): un "repartidor" que llama a cada esquema con los argumentos que necesita (LF el estado anterior; EI y CN la jacobiana).
- **`integrar_Cauchy`** (90–105): el bucle de siempre. Ahora devuelve también los tiempos `t`, y arranca LF con un paso de RK4.
- **`richardson_error`** (108–117): lo de `Cauchy_Problem.py` del Milestone 3, en una función.
- **`stability_report`** (120–131): calcula máx $|R(\Delta t\lambda)|$ para los autovalores de un eje. Para LF mira las dos raíces, la física y la parásita. Compara con $1 + 10^{-12}$, y no con 1 exacto, para que un $|R| = 1$ que sale 1.0000000000000002 por redondeo no cuente como inestable. Antes ocurría con LF en e_1.

5.3. `cuerpos.py`: el catálogo de cuerpos

La idea

Para que la animación muestre una raqueta, un libro o un asteroide, y no una caja genérica, cada cuerpo se construye con **piezas sencillas** (cajas, cilindros, conos, elipsoides), cada una con su volumen, su densidad y sus "segundos momentos" $\int x^2$, $\int y^2$ e $\int z^2$. Con ellas se calcula el tensor de inercia del conjunto en dos pasos:

1. Se suman las piezas, trasladando cada una a su sitio (**teorema de Steiner**): $C = \sum_k m_k(C_k + \mathbf{d}_k\mathbf{d}_k^T)$, donde $\mathbf{d}_k$ es la distancia de la pieza al centro de masas.
2. Se pasa de segundos momentos a inercia: $I = \mathrm{tr}(C)\,\mathrm{Id} - C$. Por ejemplo, $I_{xx} = \int(y^2 + z^2) = \mathrm{tr}(C) - C_{xx}$.

Analogía

Es como calcular el peso de un mueble de Lego sumando el de cada pieza. Steiner añade la "penalización" por estar lejos del centro: una pieza en la punta de la raqueta pesa mucho más en la inercia que la misma pieza en el mango.

```
class Cuerpo:
    """Cuerpo rigido: caras normalizadas (radio envolvente 1) e inercia."""

    def __init__(self, nombre, descripcion, piezas):
        self.nombre = nombre
        self.descripcion = descripcion
        masa = sum(p.masa for p in piezas)
        centro = sum(p.masa * p.centro for p in piezas) / masa
        segundo_momento = sum(
            p.masa * (p.covarianza + np.outer(p.centro - centro, p.centro - centro))
            for p in piezas
        ) / masa
        caras = np.concatenate([p.caras for p in piezas]) - centro
        escala = 1 / np.linalg.norm(caras.reshape(-1, 3), axis=1).max()
        self.caras = caras * escala
```

```
16          self.colores = np.concatenate([p.colores for p in piezas])
17          segundo_momento = segundo_momento * escala**2
18          # Tensor de inercia por unidad de masa: I = tr(C) Id - C.
19          self.inercia = np.trace(segundo_momento) * np.eye(3) - segundo_momento
20          self.segundos_momentos = np.diag(segundo_momento).copy()
21
22      @property
23      def momentos(self):
24          return np.diag(self.inercia).copy()
25
26      def inercias_tipicas(self, maximo=2.0):
27          """(I1, I2, I3) del cuerpo, escalados para que I3 = maximo."""
28          momentos = self.momentos / self.momentos.max() * maximo
29          return tuple(float(I) for I in momentos)
30
31      def deformar(self, I1, I2, I3):
32          """Estira el cuerpo segun cada eje para que tenga inercias (I1, I2, I3).
33
34          Los segundos momentos son Jx = (I2 + I3 - I1)/2, etc. Si alguno es
35          negativo, los momentos violan la desigualdad triangular I3 <= I1 + I2
36          y no existe ningun solido real con ellos (se dibuja casi plano).
37          """
38          J = 0.5 * np.array([I2 + I3 - I1, I1 + I3 - I2, I1 + I2 - I3])
39          # Tolerancia para cuerpos planos (I1 + I2 = I3, como la raqueta).
40          realizable = bool(J.min() > -1e-3 * J.max())
41          J = np.maximum(J, 1e-3 * J.max())
42          caras = self.caras * np.sqrt(J / self.segundos_momentos)
43          caras /= np.linalg.norm(caras.reshape(-1, 3), axis=1).max()
44          return caras, normales(caras), realizable
```

- **Líneas 141–153**: masa total, centro de masas, Steiner y paso a inercia. El cuerpo se reescala para que quepa en una esfera de radio 1.
- **`inercias_tipicas`** (160–163): escala las inercias para que $I_3 = 2$. Son los valores que se cargan en los deslizadores.
- **`deformar`** (165–178): el camino inverso. Si mueves los deslizadores de I, estira el cuerpo para que tenga *exactamente* esas inercias. Despejando, los segundos momentos son $J_x = \frac{1}{2}(I_2 + I_3 - I_1)$, etc. Si alguno sale negativo, es que se viola $I_3 \leq I_1 + I_2$ y ningún cuerpo real tiene esos momentos (la GUI avisa con $\triangle$).

5.4. `gui_reactive.py`: la GUI sencilla

No se explica línea a línea. Por bloques:

- **Ventana** (líneas 30–76): cuatro gráficas ($\omega(t)$, error de los invariantes, espacio de fases 3D y ejes del satélite), ocho deslizadores (I_1, I_2, I_3, $\omega_1(0), \omega_2(0), \omega_3(0)$, Δt, t_f), un selector de esquema (con RK4A) y un botón. Los rangos de los deslizadores obligan a $I_1 < I_2 < I_3$, y los valores iniciales ($I = (0.8, 1.5, 2.2)$, $\boldsymbol{\omega}_0 = (0.05, 1.5, 0.05)$) ya muestran un volteo.
- **Simulación "en vivo"**: un temporizador llama a `avanzar_paso` cada 10 ms, que da un paso y redibuja. El botón alterna iniciar y pausar; al reanudar, se empieza de cero.
- **Richardson sobre la marcha**, el detalle más ingenioso: en cada paso avanza *a la vez* tres simulaciones, con Δt, $\Delta t/2$ (2 subpasos) y $\Delta t/4$ (4 subpasos). Al terminar, ya tiene las tres mallas y calcula el orden y el error sin volver a integrar:

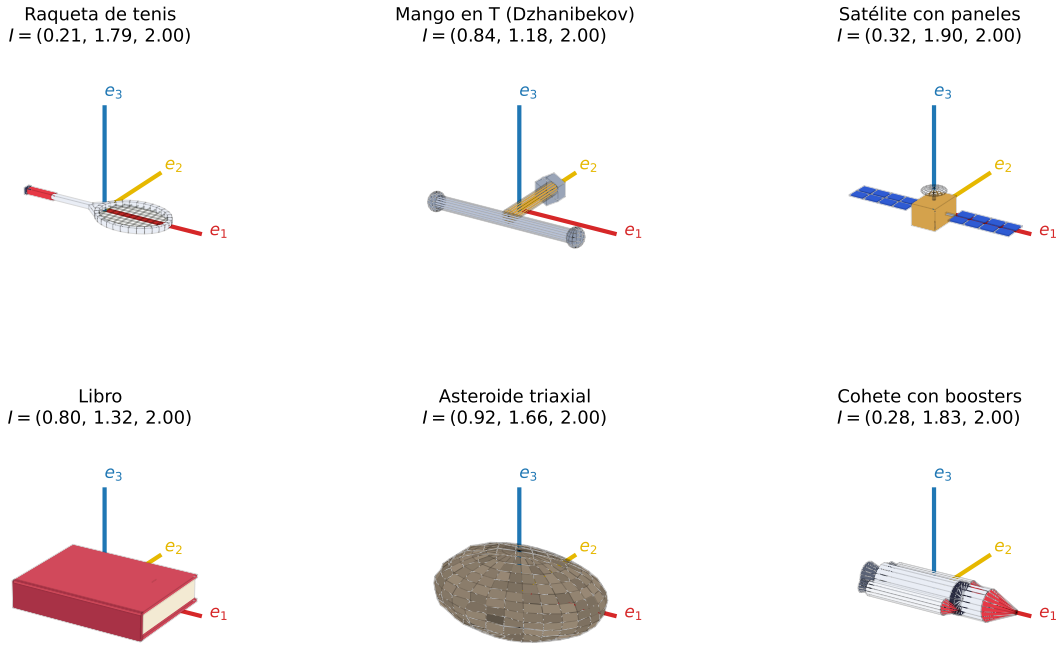


Figura 3: Los seis cuerpos del catálogo con sus ejes principales (rojo e_1 menor, amarillo e_2 intermedio, azul e_3 mayor) e inercias escaladas a $I_3 = 2$. La raqueta casi cumple $I_1 + I_2 = I_3$ porque es casi plana.

```python
    def avanzar_paso(self):
        actual = self.U[-1].copy()
        for factor, malla in zip((1, 2, 4), self.mallas):
            dt = self.At / factor
            for subpaso in range(factor):
                indice = len(malla) - 1
                tiempo = self.t[self.n] + subpaso * dt
                if self.esquema == "LF" and indice == 0:
                    siguiente = RK4(malla[-1], dt, tiempo, self.F)
                else:
                    anterior = malla[-2] if self.esquema == "LF" else None
                    siguiente = paso_Cauchy(
                        self.esquema,
                        malla[-1],
                        anterior,
                        dt,
                        tiempo,
                        self.F,
                        self.J,
                    )
                malla.append(siguiente)
        nuevo = self.U[-1]
```

- **Mensaje de estabilidad** (`texto_estabilidad`): busca el eje alrededor del que se gira más, calcula sus autovalores con la Ω real, quita el autovalor 0 y escribe si es "centro" o "silla" junto con el $|R(\Delta t\lambda)|$ del esquema. Si $I_3 > I_1 + I_2$ añade un aviso de que ningún sólido real tiene esas inercias.

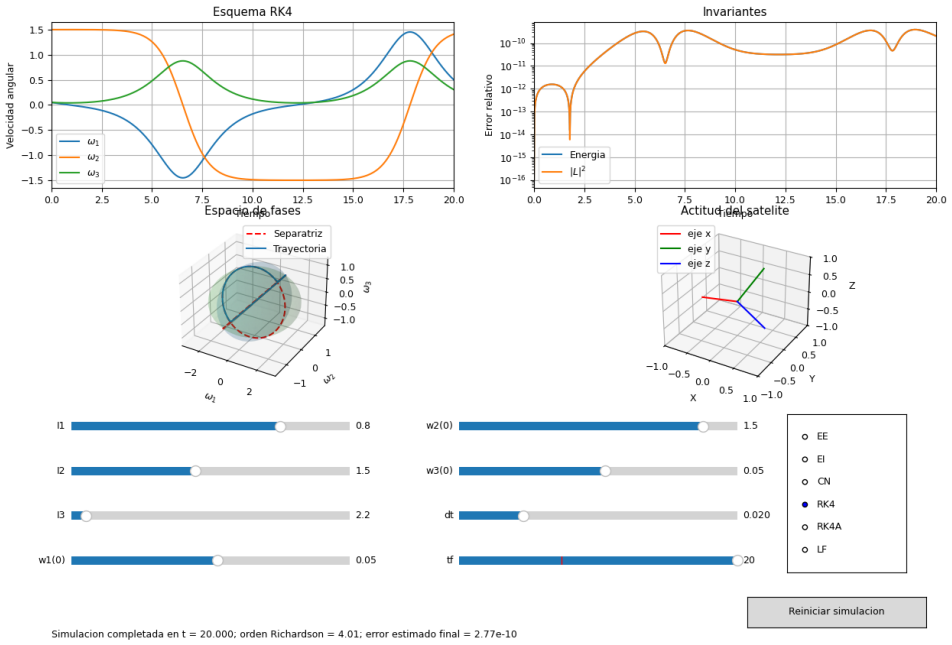


Figura 4: `gui_reactive.py` tras una simulación con RK4, $\boldsymbol{\omega}_0 = (0.05, 1.5, 0.05)$, $\Delta t = 0.02$ y $t_f = 20$. Abajo: "orden Richardson = 4.01". Las gráficas se reajustan solas y el espacio de fases muestra las dos ramas de la separatriz.

Problema encontrado: lo que estaba mal (ya corregido)

1. **Los ejes de las gráficas no se reajustaban**: se fijaban con el primer punto, y las curvas se salían (la de ω perdía la parte negativa y la de invariantes no se veía). Ahora, tras cada paso, se llama a `relim` y `autoscale_view`.
2. **$\boldsymbol{\omega}_0$ empezaba en** $(0, 0, 0)$: al pulsar "Iniciar" sin tocar nada no pasaba nada. Ahora empieza en $(0.05, 1.5, 0.05)$.
3. **Las inercias por defecto eran imposibles**: $(0.8, 1.5, 3.0)$ no cumple $I_3 \leq I_1 + I_2$. Ahora $I_3 = 2.2$.
4. **El mensaje de estabilidad** usaba siempre el eje intermedio con $\Omega = 1$ y siempre decía "inestable". Ahora usa el giro elegido (ver arriba).
5. **Solo dibujaba una rama de la separatriz**. Ahora dibuja las dos.
6. Con RK4A, al terminar dice que Richardson no aplica, en vez de dar un orden sin sentido.

5.5. `gui_actitud.py`: el cuadro de mando completo

Es la GUI que cumple todo lo que pide el enunciado. Por bloques:

- **Columna izquierda**: elección de cuerpo (6 del catálogo), del eje de giro inicial (e_1, e_2 o e_3) y del integrador (6 esquemas, con RK4A). Debajo, el **análisis espectral**: los autovalores de cada eje con el veredicto "centro, estable" o "silla, inestable"; los invariantes con la comprobación $2I_1T \leq L^2 \leq 2I_3T$; el $|R(\Delta t\lambda)|$ del esquema elegido; y el botón de Richardson.
- **Centro**: el cuerpo 3D girando, con sus ejes, $\mathbf{L}$ (morado) y $\boldsymbol{\omega}$ (verde), y un contador de volteos.
- **Deslizadores**: I_1, I_2, I_3 (siempre ordenados, porque cada uno limita a sus vecinos), Ω, ε (la perturbación), Δt, t_f y la velocidad de reproducción.
- **Derecha**: $\omega(t)$, el espacio de fases con los dos elipsoides, la separatriz, los equilibrios y la trayectoria, y la deriva de T y L^2 en escala logarítmica.

Tres ideas físicas importantes del código.

1. **El dibujo pone L vertical** (`_alinear_con_z`, línea 693). Como **L** no se mueve en el espacio, así el volteo se ve como "el eje pasa de arriba a abajo".

2. **Contador de volteos** (líneas 732–736): sigue la punta del eje de giro inicial. Cada vez que su altura cambia de lado por más de 0.5, cuenta un volteo.

3. **Detector de divergencia** (líneas 720–727): si Newton falla, aparece un NaN o $|\boldsymbol{\omega}|$ se hace mil veces mayor que al principio, para la simulación y dice "EL ESQUEMA DIVERGE". Ocurre, por ejemplo, con LF y un Δt grande.

```
    def _avanzar(self):
        """Un paso del esquema elegido y de la actitud. False si diverge."""
        n = self.n
        try:
            with np.errstate(all="ignore"):
                if self.esquema == "LF" and n == 0:
                    nuevo = RK4(self.U[0], self.At, self.t[0], self.F)
                else:
                    anterior = self.U[n - 1] if n > 0 else None
                    nuevo = paso_Cauchy(self.esquema, self.U[n], anterior,
                                        self.At, self.t[n], self.F, self.J)
        except (RuntimeError, np.linalg.LinAlgError):
            nuevo = None
        if (nuevo is None or not np.all(np.isfinite(nuevo))
                or np.linalg.norm(nuevo) > 1e3 * np.linalg.norm(self.W0)):
            self.en_marcha = False
            self.timer.stop()
            self.estado = "diverge"
            return False

        self.U[n + 1] = nuevo
        self.R = paso_actitud(self.R, self.U[n], nuevo, self.At)
        self.n = n + 1
        punta = self.P @ self.R[:, self.eje]
        self.puntas[self.n] = punta
        if punta[2] * self._lado < -0.5:
            self._lado = -self._lado
            self.volteos += 1
        return True
```

Una idea de software: el *blitting*. Redibujar toda la ventana cuesta medio segundo, demasiado para una animación. El código dibuja una vez el fondo estático (marcos, ejes, textos fijos), lo guarda como imagen, y en cada fotograma solo repinta lo que se mueve: el cuerpo, las curvas y el reloj.

Analogía

Es como los dibujos animados clásicos: el decorado se pinta una vez y encima se mueven solo los personajes, dibujados en hojas transparentes.

Problema encontrado: no arrancaba con matplotlib 3.10 (ya corregido)

Con la versión instalada en este ordenador (matplotlib 3.10.8), `PanelActitud()` fallaba al crearse. Metía en una `Line3DCollection` un círculo de 90 puntos y una recta de 2, y al autoescalar, `add_collection3d` intenta convertirlos en una sola matriz: `ValueError: setting an array element with a sequence...` Por eso fallaba 1 de los 10 tests. **Arreglo**: la recta vertical de referencia se dibuja ahora con 90 puntos, como el círculo (línea 368). Así funciona con cualquier versión de matplotlib, sin depender de opciones nuevas.

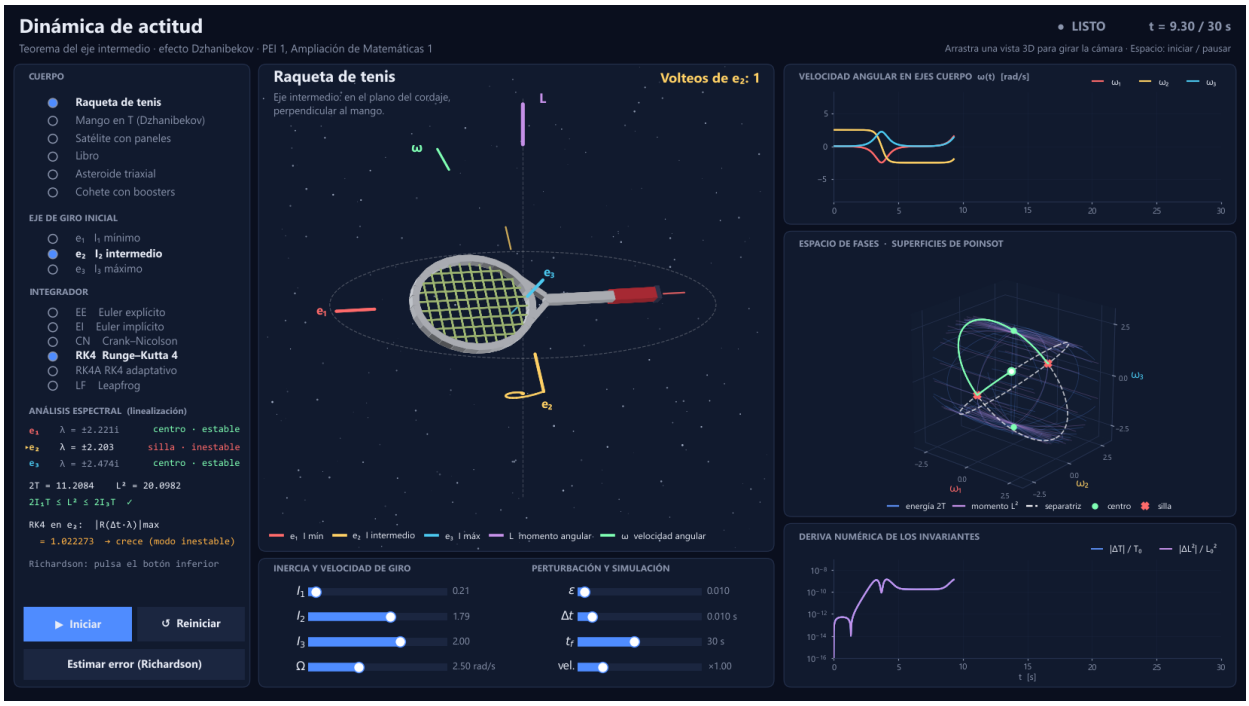


Figura 5: `gui_actitud.py` con los valores por defecto (raqueta, giro en e_2, RK4), detenida en $t = 9.3$ en mitad del segundo volteo. Arriba: "Volteos de e_2: 1".

Problema encontrado: el autovalor 0 escondía la disipación de EI (ya corregido)

El panel calculaba máx $|R(\Delta t\lambda)|$ sobre *los tres* autovalores del eje, incluido $\lambda = 0$, y $R(0) = 1$ en todos los esquemas. En un centro con EI, los autovalores transversales dan $|R| = 0.99975 < 1$ (disipa), pero el máximo de los tres era el 1 del autovalor cero, y el panel decía **"= 1.000000 → conserva"**, que es falso. **Arreglo**: antes de llamar a `stability_report` se quitan los autovalores nulos (`lam = lam[np.abs(lam) > 1e-12]`). Un test nuevo comprueba que en e_1 el panel dice "amplifica" con EE, "disipa" con EI y "conserva" con CN y LF.

RK4A en el cuadro de mando. Aparece en el selector de integrador. Como su error lo fija la tolerancia y no Δt, el botón de Richardson responde "no aplica a RK4A" en lugar de dar un orden sin sentido.

5.6. `test_solution.py`: las pruebas automáticas

Un test es un pequeño programa que comprueba que otro hace lo que debe. Se ejecutan todos con `python -m unittest PEI_1.Solucion_PEI1.test_solution -v` desde la raíz del repositorio.

Test	Qué comprueba	Resultado
`ecuaciones_y_jacobiano`	$\mathbf{F}$ en un punto; la jacobiana analítica coincide con la numérica	OK
`estabilidad_de_los_ejes`	e_1, e_3 con autovalores imaginarios y e_2 real	OK
`integradores_y_invariantes`	los 6 esquemas dan resultados finitos	OK
`separatriz_y_richardson`	la separatriz cumple $L^2 = 2I_2T$; RK4 de orden > 3.5	OK
`funciones_de_estabilidad`	$R(0) = 1$; RK4 estable en $z = 0.1i$; CN y LF estables con redondeo	OK
`rk4_adaptativo` *(nuevo)*	RK4A 100 veces más preciso que RK4 con el mismo Δt	OK
`panel_espectral_distingue...` *(nuevo)*	en e_1: EE amplifica, EI disipa, CN y LF conservan	OK
`gui_reactive_reajusta...` *(nuevo)*	los ejes contienen las curvas y hay volteo	OK
`actitud`	R sigue siendo una rotación ($RR^T = I$, $\det = 1$)	OK
`pausar_conserva_graficas...`	pausa y reinicio de `gui_reactive.py`	OK
`catalogo_de_cuerpos`	ejes principales ordenados, $I_3 \le I_1 + I_2$, `deformar`	OK
`cotas_de_la_polhodia`	la trayectoria no supera las cotas teóricas de $\|\omega_k\|$	OK
`panel_actitud_voltea...`	solo voltea en e_2 (0, ≥ 2, 0 volteos)	OK (antes ERROR)

Antes de las correcciones había 10 tests y 1 fallaba: el panel no llegaba a crearse por lo de matplotlib 3.10. Ahora hay 13 y pasan todos.

6. Experimentos con el caso por defecto

Caso por defecto de `gui_actitud.py`: raqueta $I = (0.211, 1.793, 2.000)$, giro en e_2 con $\Omega = 2.5$, $\varepsilon = 0.01$, $\Delta t = 0.01$ y $t_f = 30$. Se ha integrado con los cinco esquemas de paso fijo de `Solucion_PEI1/numerical_engine.py` y se ha comparado con una solución de referencia muy precisa. (RK4A tiene su propia tabla en la sección 3.6.)

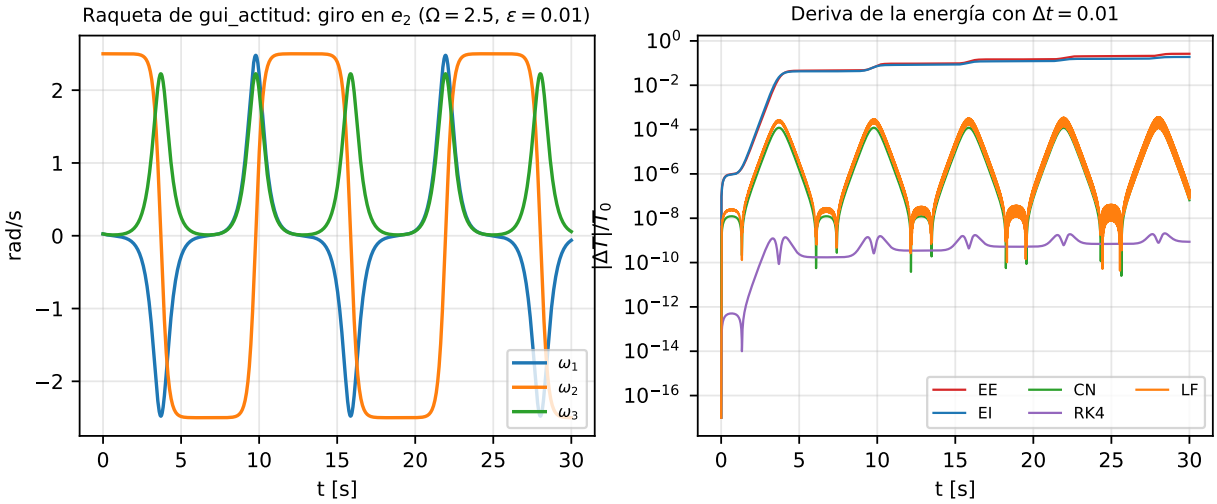


Figura 6: Izquierda: $\boldsymbol{\omega}(t)$ de la raqueta, con cinco volteos en 30 s. Derecha: error relativo de la energía con cada esquema. EE y EI se alejan para siempre a escalones (uno por volteo). CN y LF oscilan, sin tendencia a crecer. RK4 se queda en 10^{-9}.

Lo que dice la simulación

	EE	EI	CN	RK4	LF
máximo $\lvert\Delta T\rvert/T_0$	0.26	0.19	1.2×10^{-4}	2.1×10^{-9}	3.7×10^{-4}
$\lvert\Delta T\rvert/T_0$ en $t = 30$	0.26	0.19	6.5×10^{-8}	8.7×10^{-10}	1.9×10^{-7}
error de $\boldsymbol{\omega}$ en $t = 30$	0.31	0.25	7.2×10^{-5}	1.1×10^{-8}	1.4×10^{-4}
último volteo (ref. 28.01)	28.11	27.92	28.01	28.01	28.01
tiempo de cálculo	0.01 s	0.07 s	0.09 s	0.04 s	0.01 s
orden (Richardson, $t_f = 10$)	1.01	0.99	2.00	3.98	2.00
orden (Richardson, $t_f = 30$)	1.08	0.93	2.00	3.95	2.00

Cómo leer la tabla:

- **EE crea energía (+26 %) y EI la destruye (−19 %)**, como en el oscilador y por el mismo motivo (Pitágoras). El error se acumula en los volteos, cuando **F** es grande. EE llega tarde al último volteo y EI pronto.
- **CN y LF tienen un error que sube y baja, pero no crece**. Son métodos *simétricos en el tiempo*, y estos no tienen deriva sistemática. Al final de cada volteo la energía vuelve casi a su valor. Ojo: CN *no* conserva aquí la energía exactamente, porque el problema no es lineal. El que la conservaría exactamente es el punto medio implícito.
- **RK4 es el más preciso** con mucho, y cuesta menos que EI y CN porque no resuelve ecuaciones.
- **LF es tan barato como EE** (una evaluación de **F** por paso) y cuatro órdenes de magnitud más preciso. Eso sí, solo funciona bien en problemas sin rozamiento.

- Los órdenes medidos coinciden con los teóricos (1, 1, 2, 4, 2). EE y EI se desvían algo con $t_f = 30$ porque el error ya es grande, que es lo que vimos en el Milestone 3.

7. Frente al enunciado y cambios hechos en el código

7.1. Lo que pide el enunciado

Pide el enunciado	Estado en `Solucion_PEI1`
F, jacobiana exacta, $2T$ y L^2	hecho (`space_physics.py`)
Euler, CN, RK4	hecho, más EI y LF
Paso **adaptativo**	hecho: RK4A (sección 3.6), añadido en esta revisión
Richardson	hecho (función, botón en `gui_actitud.py` y mallas simultáneas en `gui_reactive.py`)
$\|R(z)\| \leq 1$	hecho (`estabilidad`, `stability_report`)
Deslizadores I, Ω, Δt; cambio de esquema	hecho en `gui_actitud.py`
Fases 3D, Poinsot, separatriz, actitud 3D	hecho en `gui_actitud.py`
Tres capas desacopladas	hecho (las flechas de importación van solo hacia abajo)

7.2. Cambios hechos en el código

1. **Base** `space_physics.py`: índices corregidos, productos correctos, devuelve un vector nuevo; `invariantes` compara L^2 (no L) y devuelve $(2T, L^2)$; fuera el truco del `sys.path`.

2. **Base** `numerical_engine.py`: fuera la importación rota de `Milestone_3`; LF arranca con RK4 (orden medido 2.00 en vez de 2.43).

3. **Base** `gui_reactive.py`: inercias (0.010, 0.014, 0.020) en vez de una moneda.

4. `numerical_engine.py` (solución): nuevo esquema adaptativo `RK4A`; `stability_report` con tolerancia de redondeo y sin fallar si no hay autovalores.

5. `space_physics.py` (solución): $R(z)$ para RK4A.

6. `gui_actitud.py`: arranca con matplotlib 3.10; quita el autovalor 0 en el análisis espectral; RK4A en el selector; Richardson avisa de que no aplica a RK4A.

7. `gui_reactive.py` (solución): ejes que se reajustan; $\boldsymbol{\omega}_0$ e inercias por defecto que muestran un volteo y son realizables; mensaje de estabilidad con el giro real y aviso si $I_3 > I_1 + I_2$; las dos ramas de la separatriz; RK4A.

8. `test_solution.py`: RK4A en la prueba de integradores y tres tests nuevos. Pasan los 13.

9. `README.md`: menciona RK4A.

Lo único que queda es la errata del enunciado ($I_3\dot{\omega}_1 \to I_3\dot{\omega}_3$ en la ecuación (2)), que no es código.

8. Preguntas probables del tribunal

Pregunta del tribunal: ¿por qué es inestable el eje intermedio?

Porque al linealizar alrededor de $(0, \Omega, 0)$ las dos perturbaciones se alimentan entre sí: $\delta\ddot{\omega}_1 = \frac{(I_3-I_2)(I_2-I_1)}{I_1 I_3}\Omega^2\,\delta\omega_1$. El coeficiente es positivo solo cuando I_2 está entre I_1 e I_3, así que hay un autovalor real positivo y la perturbación crece exponencialmente (punto de silla).

Pregunta del tribunal: si es inestable, ¿por qué el cuerpo vuelve en lugar de seguir alejándose?

Porque se conservan $2T$ y L^2: el estado está obligado a moverse sobre la intersección de dos elipsoides, que es una curva cerrada. La linealización solo vale mientras la perturbación es pequeña; después mandan los invariantes.

Pregunta del tribunal: ¿la linealización demuestra que e_1 y e_3 son estables?

No del todo: sus autovalores son imaginarios puros y Hartman–Grobman no se aplica. La estabilidad en el sistema no lineal la garantizan las leyes de conservación, porque cerca de esos ejes las polhodias son curvas cerradas pequeñas.

Pregunta del tribunal: ¿qué es el autovalor cero?

Es la dirección del propio eje de giro: cambiar un poco Ω lleva a otro giro puro, que también es equilibrio. Es neutro: ni crece ni decrece.

Pregunta del tribunal: ¿por qué Euler explícito hace crecer la energía?

Porque la física conserva $2T$, así que $\nabla(2T)\cdot\mathbf{F} = 0$: el paso de Euler es "perpendicular" a la dirección en que cambiaría la energía. Al dar un paso recto queda solo el término de segundo orden, $\Delta t^2 \sum I_k F_k^2 > 0$, como el Pitágoras del oscilador.

Pregunta del tribunal: ¿por qué CN no conserva aquí la energía exactamente, si en el oscilador sí?

Porque CN conserva exactamente las cantidades cuadráticas solo cuando $\mathbf{F}$ es lineal. Las ecuaciones de Euler tienen productos $\omega_i\omega_j$. El punto medio implícito, $\mathbf{U}^{n+1} = \mathbf{U}^n + \Delta t\,\mathbf{F}(\frac{\mathbf{U}^n+\mathbf{U}^{n+1}}{2})$, sí conserva exactamente $2T$ y L^2 en cualquier caso. En el oscilador los dos métodos coinciden.

Pregunta del tribunal: ¿qué ventajas y riesgos tiene Leapfrog?

Ventajas: orden 2 con una sola evaluación de $\mathbf{F}$ por paso, conserva la amplitud de las oscilaciones si $\Delta t|\lambda| \le 1$, y no tiene deriva de energía. Riesgos: necesita un arranque con otro método, tiene una raíz parásita y su región de estabilidad es solo el segmento $[-i, i]$, así que con rozamiento o con Δt grande explota.

Pregunta del tribunal: en la silla, todos los esquemas dan $|R| > 1$. ¿Son todos inestables?

No es un problema del esquema: la solución exacta crece como $e^{\sigma t}$. Lo que se le pide al método es crecer *al ritmo correcto* ($e^{\sigma\Delta t}$ por paso), no amortiguar. Con $\Delta t = 0.01$, RK4 da 1.022273 frente al exacto 1.022273.

Pregunta del tribunal: ¿cómo funciona el paso adaptativo?

Por duplicación del paso: cada subpaso se da entero y en dos mitades con RK4. Como RK4 es de orden 4, la diferencia entre 15 estima el error local (la idea de Richardson). Si es menor que la tolerancia se acepta, y el siguiente paso se escala con $(\text{tol}/\text{error})^{1/5}$; si no, se repite más corto. Así da pasos largos en las mesetas y cortos en los volteos. Su error lo fija la tolerancia, no Δt, y por eso no tiene sentido medirle un orden con Richardson.

Pregunta del tribunal: ¿cómo se calcula la orientación y por qué no se deforma?

Con rotaciones de Rodrigues: en cada paso se gira exactamente un ángulo $|\bar{\boldsymbol{\omega}}|\Delta t$ alrededor de $\bar{\boldsymbol{\omega}}$. Componer rotaciones da siempre una rotación, así que $R^T R = I$ se mantiene hasta el redondeo (7×10^{-15}). No hace falta normalizar como con cuaterniones y Euler explícito.

Pregunta del tribunal: ¿qué pasa con un cuerpo con dos inercias iguales?

No hay eje intermedio. Es un cuerpo simétrico, como una peonza o una moneda (los valores de la versión base), y no hay efecto Dzhanibekov. Al girar fuera de su eje de simetría hace una precesión regular, $\boldsymbol{\omega}$ dando vueltas a velocidad constante alrededor de ese eje, pero no voltea.

Pregunta del tribunal: ¿qué significa la cota $2I_1T \le L^2 \le 2I_3T$?

Que $L^2/2T$ es una media ponderada de las inercias y por tanto está entre la menor y la mayor. Si está por debajo de I_2 el movimiento rodea e_1, si está por encima rodea e_3, y si es igual a I_2 está sobre la separatriz.